a) Implementation of Stack using array representation and devise the operations.

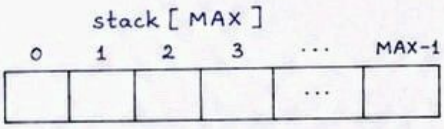
Stack :

Stack is a linear data structure which follows LIFO (Last In First Out) principle.

Example : Stack of plates, pile of books etc.

Array Representation :

We can implement stack using an array.



top = -1 (stack is empty)

3. PEEK (Display top element) :

```
int peek()
{
   if (top == -1)
   {
      printf("Stack is Empty");
      return -1;
   }
   else
      return stack[top];
}
```

Operations on Stack :

1. PUSH (Insert) :

- Check if stack is full
- Increment top
- Insert element

```
void push(int item)
{
   if (top == MAX-1)
      printf("Stack Overflow");
   else
   {
      top = top + 1;
      stack[top] = item;
   }
}
```

2. POP (Delete) :

- Check if stack is empty
- Store top element
- Decrement top

```
int pop()
{
   int item;
   if (top == -1)
   {
      printf("Stack Underflow");
      return -1;
   }
   else
   {
      item = stack[top];
      top = top - 1;
      return item;
   }
}
```

4. DISPLAY (Display all elements) :

```
void display()
{
   int i;
   if (top == -1)
      printf("Stack is Empty");
   else
   {
      printf("Stack elements : ");
      for(i = top; i >= 0; i--)
         printf("%d ", stack[i]);
   }
}
```

Advantages :

- Simple to implement
- Fast access

Disadvantages :

- Fixed size
- Stack overflow may occur

b) Evaluation of Postfix arithmetic expression using stack. Give suitable example.

Postfix expression :

In postfix expression, operator occurs after operands.

Example : AB+ → A+B

Algorithm :

1. Scan the expression from left to right.
2. If the symbol is an operand, push it into stack.
3. If the symbol is an operator, pop two operands, apply the operator and push the result back into stack.
4. After scanning entire expression, the result will be the only element left in the stack.

Example : Evaluate 23*54*+9-

Symbol	Action	Stack (bottom → top)
2	Push 2	2
3	Push 3	2 — 3
*	Pop 3,2 → 2*3=6, Push 6	6
5	Push 5	6 5
4	Push 4	6 5 4
*	Pop 4,5 → 5*4=20, Push 20	6 20
+	Pop 20,6 → 6+20=26, Push 26	26
9	Push 9	26 9
-	Pop 9,26 → 26-9=17, Push 17	17

Result = 17

C Program :

```
#include <stdio.h>
#include <ctype.h>
int stack[20], top=-1;
void push(int x)
 { stack[++top]=x; }
int pop()
 { return stack[top--]; }
int main()
{ char exp[20]; int i,a,b,res;
  printf("Enter postfix expr: ");
  scanf("%s",exp);
  for(i=0; exp[i]!='\0'; i++)
  { if(isdigit(exp[i]))
      push(exp[i]-48);
    else
    { b=pop(); a=pop();
      switch(exp[i])}
      {case '+': push(a+b); break;
       case '-': push(a-b); break;
       case '*': push(a*b); break;
      {case '/': push(a/b); break;}}}
  res = pop();
  printf("Result = %d", res);
  return 0; }
}
```

Applications of Stack :

1. Expression evaluation
2. Function calls (recursion)
3. Parenthesis checking
4. Undo operation in editors
5. Backtracking

a. QUEUE

2. question

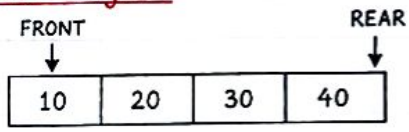
Definition :

A Queue is a linear data structure that follows FIFO (First In First Out).

- Element inserted first will be deleted first.
- Insertion operation is called En-Queue.
- Deletion operation is called De-Queue.

Example : People standing in a ticket line.

Queue Diagram



En-Queue Operation Algorithm

Algorithm : ENQUEUE(Q, ITEM)

```
Step 1 : Check if REAR = MAX-1
         If yes, print "Queue Overflow"
Step 2 : Else
             If FRONT = -1
                FRONT = 0
Step 3 : REAR = REAR + 1
Step 4 : Q[REAR] = ITEM
Step 5 : Stop
```

De-Queue Operation Algorithm

Algorithm : DEQUEUE(Q)

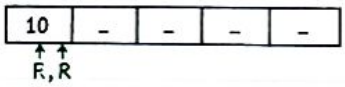
```
Step 1 : Check if FRONT = -1 OR FRONT > REAR
         If yes, print "Queue Underflow"
Step 2 : Else
             ITEM = Q[FRONT]
Step 3 : FRONT = FRONT + 1
Step 4 : Return ITEM
Step 5 : Stop
```

Trace Example

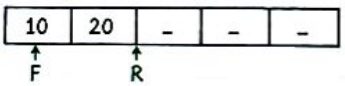
Assume Queue size = 5

Initial :
FRONT = -1
REAR = -1

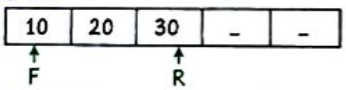
① Enqueue 10



② Enqueue 20

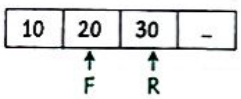


③ Enqueue 30

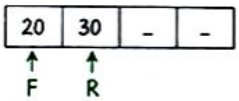


④ Dequeue

Deleted element = 10



Remaining Queue :

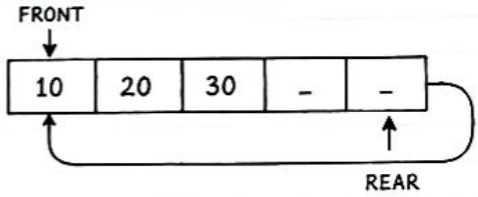


b. CIRCULAR QUEUE

Definition :

A Circular Queue is a queue in which the last position is connected back to the first position.
It avoids wastage of memory space.

Circular Queue Diagram



En-Queue Algorithm for Circular Queue

Algorithm : CIRCULAR_ENQUEUE(Q, ITEM)

```
Step 1 : Check if (REAR + 1) % MAX = FRONT
         If yes, print "Queue Overflow"
Step 2 : Else if FRONT = -1
                 FRONT = 0
                 REAR  = 0
Step 3 : Else
                 REAR = (REAR + 1) % MAX
Step 4 : Q[REAR] = ITEM
Step 5 : Stop
```

De-Queue Algorithm for Circular Queue

Algorithm : CIRCULAR_DEQUEUE(Q)

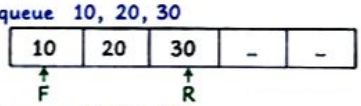
```
Step 1 : Check if FRONT = -1
         If yes, print "Queue Underflow"
Step 2 : ITEM = Q[FRONT]
Step 3 : If  FRONT = REAR
             FRONT = -1
             REAR  = -1
Step 4 : Else
             FRONT = (FRONT + 1) % MAX
Step 5 : Return ITEM
Step 6 : Stop
```

Trace Example for Circular Queue

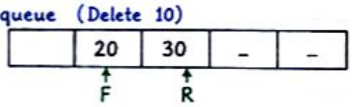
Assume Queue size = 5

Initial :
FRONT = -1
REAR = -1

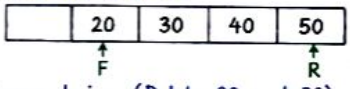
① Enqueue 10, 20, 30



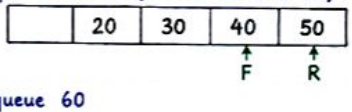
② Dequeue (Delete 10)



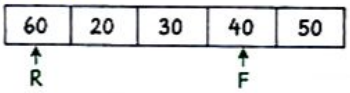
③ Enqueue 40 and 50



④ Dequeue twice (Delete 20 and 30)



⑤ Enqueue 60



Queue elements in order (from FRONT to REAR) :

40, 50, 60

FRONT = 3, REAR = 0

a. Operations on Circular Queue 3. question

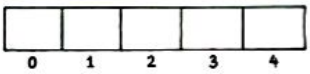
Circular Queue : Last position is connected back to first; uses modulo (%) for wrap-around.

List of Operations

No.	Operation	Description
①	Initialize	Set FRONT = -1, REAR = -1
②	En-Queue	Insert element at REAR
③	De-Queue	Delete element from FRONT
④	Peek/Front	Access element at FRONT without removing
⑤	IsEmpty	Check if FRONT = -1
⑥	IsFull	Check if (REAR+1) % MAX = FRONT
⑦	Display	Print all elements from FRONT to REAR

① Initialize

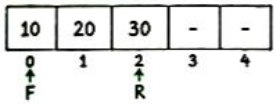
FRONT = -1, REAR = -1



② En-Queue (Insert)

1. If (REAR + 1) % MAX = FRONT
   → Queue Overflow
2. Else if FRONT = -1
   • FRONT = 0, REAR = 0
3. Else REAR = (REAR + 1) % MAX
4. Q[REAR] = ITEM

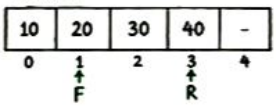
MAX=5



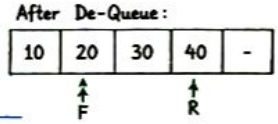
③ De-Queue (Delete)

1. If FRONT = -1 → Underflow
2. ITEM = Q[FRONT]
3. If FRONT = REAR
   • FRONT = -1, REAR = -1
4. Else FRONT = (FRONT + 1) % MAX

Before:



After De-Queue:



④ Peek/Front

• Return Q[FRONT] without removing

Q[FRONT] = 20

⑤ IsEmpty

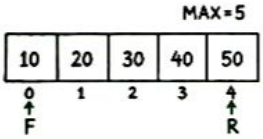
Condition: FRONT = -1
→ Queue is empty



⑥ IsFull

Condition: (REAR + 1) % MAX = FRONT
→ Queue is full

MAX=5



⑦ Display

Print elements from FRONT to REAR

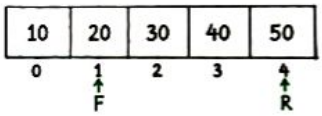
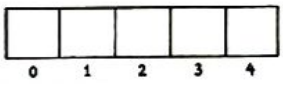
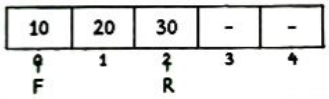


Diagram of Operations (MAX = 5)

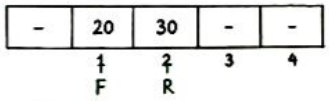
Initial:
F = -1, R = -1



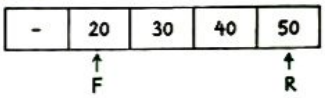
① Enqueue 10, 20, 30



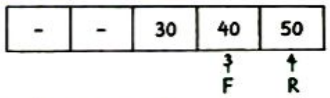
② Dequeue (Delete 10)



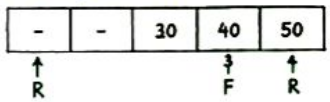
③ Enqueue 40, 50



④ Dequeue twice (Delete 20, 30)



⑤ Enqueue 60 (Wrap-around)



Elements in queue (from F to R):

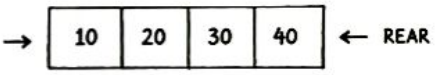
40, 50, 60

b. Types of Queues (Any Two)

① Simple Queue (Linear Queue)

- Follows FIFO (First In First Out)
- Insertion at REAR, Deletion from FRONT
- Basis for most queue applications

Diagram:



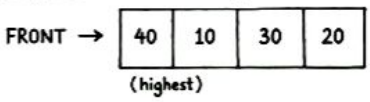
Example:

- Printer queue
- CPU job scheduling

② Priority Queue

✓ Each element has a priority
✓ Higher priority element is served first
✓ Can be Ascending (min) or Descending (max)

Diagram (Max Priority):



Priority

Item	Priority
A	3
B	1
C	4
D	2

Example:

- Hospital emergency patients
- OS task scheduling

Dequeue order:

C, A, D, B

a. Infix to Postfix Conversion using Stack

4. question

Infix Expression :

A + (B * C - (D / E ^ F) * G) * H

Operator Precedence :

^ → Highest
* , / → Next
+ , - → Lowest

Step-by-Step Trace :

Step	Scan Symbol	Stack Contents	Postfix Expression
1	A	-	A
2	+	+	A
3	(	+ (	A
4	B	+ (	AB
5	*	+ (*	AB
6	C	+ (*	ABC
7	-	+ (-	ABC *
8	(	+ (- (	ABC *
9	D	+ (- (	ABC * D
10	/	+ (- (/	ABC * D
11	E	+ (- (/	ABC * DE
12	^	+ (- (/ ^	ABC * DE
13	F	+ (- (/ ^	ABC * DEF
14	)	+ (-	ABC * DEF ^ /
15	*	+ (- *	ABC * DEF ^ /
16	G	+ (- *	ABC * DEF ^ / G
17	)	+	ABC * DEF ^ / G * -
18	*	+ *	ABC * DEF ^ / G * -
19	H	+ *	ABC * DEF ^ / G * - H
20	End	-	ABC * DEF ^ / G * - H * +

Final Postfix Expression :

ABC * DEF ^ / G * - H * +

b. Evaluation of Postfix Expression using Stack

Algorithm : EVALUATE_POSTFIX (P)

Step 1 : Start
Step 2 : Scan postfix expression P from left to right.
Step 3 : If symbol is an operand,
PUSH it into stack.
Step 4 : If symbol is an operator,
POP operand2 from stack.
POP operand1 from stack.
Perform operation operand1 operator operand2.
PUSH the result into stack.
Step 5 : Repeat Steps 2 to 4 until end of expression.
Step 6 : POP the final value from stack.
This is the result.
Step 7 : Stop

Postfix Expression : ABC*+D+

Given : A = 2, B = 3, C = 4, D = 5

Step-by-Step Evaluation :

Step	Scan Symbol	Operation	Stack Contents (Bottom → Top)	Explanation
1	A	Push 2	2	A = 2
2	B	Push 3	2, 3	B = 3
3	C	Push 4	2, 3, 4	C = 4
4	*	Pop 4, 3 3 * 4 = 12 Push 12	2, 12	Multiply
5	*	Pop 12, 2 2 + 12 = 14 Push 14	14	Add
6	D	Push 5	14, 5	D = 5
7	*	Pop 5, 14 14 + 5 = 19 Push 19	19	Add

Final Result (value in stack) = 19

Stack Diagram :

